# TAPLab: An Open Laboratory for Reproducible Traffic Assignment Experiments

## Design Principles, Independent Certification, and Cross-Solver Evidence

**Xuesong (Simon) Zhou, Ph.D.** (Corresponding Author)
Professor, School of Sustainable Engineering and the Built Environment
Arizona State University, Tempe, AZ 85281
Email: xzhou74@asu.edu

[Co-authors to be added with job titles and affiliations]

Word count: [to be computed] words + [n] tables $\times$ 250 = [total]
Total number of pages: [n]

Submitted for presentation at the Transportation Research Board Annual Meeting and publication in Transportation Research Record.

*Disclosure: generative AI tools were used to assist code development, experiment orchestration, and manuscript drafting under full author direction and review; all algorithmic designs, definitions, and conclusions are the authors'.*

# Abstract

**Objectives.** Computational comparisons of traffic assignment algorithms are difficult to trust and reproduce: solvers report their own convergence measures, format conversions silently alter the problem, and restricted path-pool results are presented as network equilibria. This paper introduces TAPLab, an open laboratory that makes traffic assignment experiments reproducible and independently verifiable across solvers.

**Methods.** TAPLab combines (i) a compact GMNS-compatible exchange representation; (ii) TAPForge, a parametric instance generator whose manifests reproduce every instance from recorded parameters and seeds; (iii) solver adapters for link-based, bush-based, origin-based, path-based, and compressed-path algorithms spanning five independent codebases; and (iv) an independent certification pipeline that recomputes link costs, the Beckmann objective, flow conservation, and the shortest-path gap from returned flows, never trusting solver self-reports.

**Findings.** The certification pipeline exposed five latent defects in standard interchange practice, each invisible to self-reported gaps, including centroid through-routing from first-through-node defaults and a capacity value colliding with a solver's internal sentinel. A certified seven-algorithm comparison across analytical, route-rich, space-time, and canonical networks confirms bush-based methods dominate one-shot solving, while controlled same-kernel isolation shows when and why path-space compression pays.

**Novelty.** TAPLab is, to our knowledge, the first traffic assignment platform in which every published number carries an independent certificate and every generated instance is reproducible from its manifest alone.

**Practical Applications.** Agencies can verify vendor solver claims; researchers can attribute performance differences to algorithms rather than implementations; solver developers gain regression laboratories with transparent failure reporting.

# 1 Introduction

Static traffic assignment underpins regional planning, project evaluation, and pricing studies, and a rich family of algorithms now solves it: link-based Frank–Wolfe variants, bush-based methods in the lineage of Algorithm B and TAPAS, origin-based schemes such as LUCE, path-based gradient projection, and recent compressed-path representations. Yet published computational comparisons remain difficult to reproduce or even to interpret . . . [reproducibility motivation; the three failure modes: self-reported gaps, conversion drift, restricted-pool conflation].

This paper presents TAPLab, an open laboratory developed over a multi-year research program and progressively refined toward a fully reproducible workflow pipeline. Contributions: (1) the design-principle framework; (2) the architecture and its certified adapter suite; (3) empirical evidence from a defect study and a certified seven-algorithm comparison; (4) three demonstration studies showing the experimental protocols the laboratory enables.

# 2 Design Principles

The laboratory is defined by eight principles, stated here as requirements that any reproducible assignment experiment should satisfy.

**P1 — Independent certification.** Every published number is recomputed from the solver's returned link flows: BPR costs, the Beckmann objective, node-level flow conservation against the OD table, and the all-or-nothing shortest-path lower bound yielding the relative gap. A solver's self-reported gap must agree with the recomputation within an order of magnitude, or the run is not certified.

**P2 — Restricted-master gaps are not network claims.** A fixed path-pool solve reports a pool gap; the full-network gap is recomputed separately, and only network-wide pricing certifies equilibrium.

**P3 — Reproducibility by manifest.** Every generated instance records its complete parameter set and seed; every network statistic is computed from the instance, never hard-coded in documentation.

**P4 — Same-kernel isolation.** Representation effects are measured with the routing kernel held fixed, so a "solver improvement" cannot be a shortest-path implementation improvement in disguise.

**P5 — Transparent failure reporting.** Timeouts, crashes, and empty results appear as rows in every result table.

**P6 — Work-unit accounting.** Time-to-verified-accuracy and algorithm-appropriate work counters (sweeps, shortest-path calls, oracle calls) complement wall-clock time; algorithms with different work units are never ranked by runtime alone.

**P7 — Precise object definitions.** Distinct mathematical objects receive distinct names; in particular, a path-group compression column and a complete feasible origin-flow template are never conflated.

**P8 — Provenance separated from license.** Code licensing (MIT) never speaks for data; each

bundled network carries its own provenance, citation, terms, and content checksums.

# 3 Laboratory Architecture

[Components: TAPBench (tiered instances), TAPForge (parametric generator: analytical, Manhattan-grid A2/A3, origin-structure A1, space–time A4), TAPRunner (adapters over five codebases: tap-b, TAsK, TAPLite, AequilibraE, the CompressedTAP column engine, plus built-in reference solvers), TAPValidate (input invariants + computed statistics + the certification pipeline), TAPView / TAP-Dashboard (inspection), TAPReports. Compact GMNS: centroids as node rows, connectors as typed links, gzip-transparent loading. CLI workflow figure.]

# 4 What Certification Catches: A Defect Study

Five defects in standard interchange practice, found by certification and invisible to self-reported gaps (Table 1).

**Table 1:** Interchange defects exposed by independent certification. Each produced a plausible-looking self-reported gap.

| Defect | Symptom if uncaught | Detection |
|---|---|---|
| First-through-node default lets paths route through centroids | Anaheim flows 72% RMSE from best-known at a self-reported gap of $5 \times 10^{-7}$ | Certified gap vs. best-known flows |
| Connector capacity equal to a solver's internal artificial-arc sentinel | All connector flows silently missing from output | Conservation residual of 10,710 trips |
| Fixed-decimal export precision | Braess $t_0 = 10^{-8}$ became 0; the wrong equilibrium became optimal for the corrupted network | Recomputed gap 0.24 vs. self-reported 0 |
| Links not grouped in forward-star order | Parser over-allocates nodes; solver aborts or corrupts | Transparent failure row |
| Origin absent from trip-table sequence | OD parser segmentation fault | Bisection to the missing zone |

# 5 A Certified Seven-Algorithm Comparison

[Arms: Algorithm B (tap-b), TAPAS, LUCE, BFW (TAsK), adaptive latent-GP, and two fixed-pool C++ arms (full GP, latent-atom), plus a projected- gradient/reduced-gradient/latent decomposition row per instance. Instance ladder: analytical, route-rich grids $n$=5..40, space–time DAGs, Sioux Falls, Chicago Sketch. Race figure.]

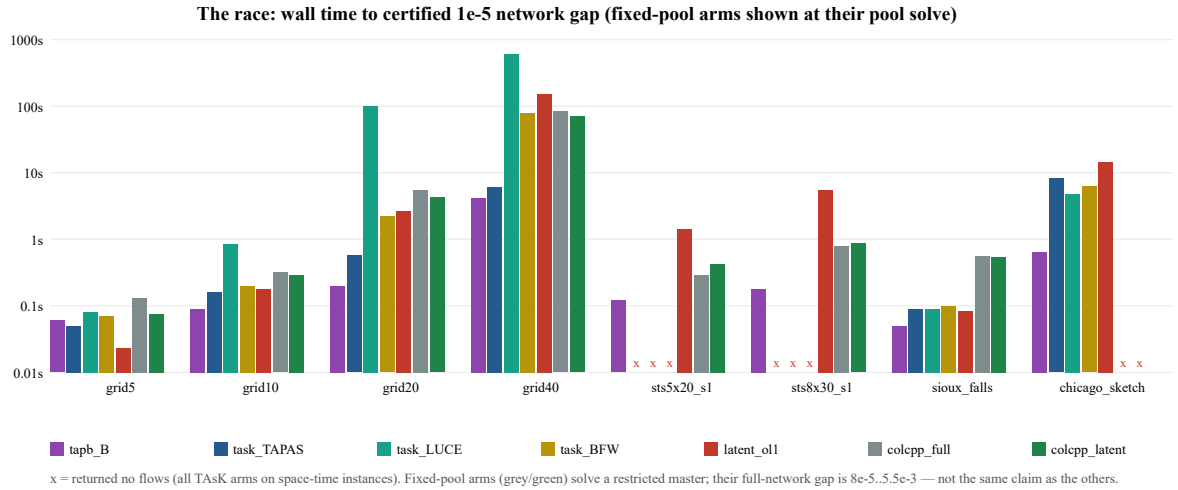


**Figure 1:** Certified wall time to a $10^{-5}$ network gap across the instance ladder. Failures remain visible; fixed-pool arms are flagged as making a restricted-master claim (P2, P5).

[Headline findings: bush methods dominate one-shot solving at scale (grid40: B 4.2 s, TAPAS 6.0 s, BFW 79.9 s, LUCE at cap); origin-based LUCE degrades sharply on route-rich congested grids; only two arms return solutions on space–time instances.]

# 6 Demonstration Studies

## 6.1 Same-kernel isolation of a representation effect (P4, P7)

[The latent-atom compression as the worked example: same engine, same pool, atom on/off; Figure 2; speedups 1.05–1.7× end-to-end at $10^{-6}$-level objective error; effect grows with path richness; k-sweep shows over-compression fails by churn, not accuracy.]

## 6.2 Restricted-pool adequacy pricing (P2)

[K=32 penalty-KSP pools solve to $10^{-5}$ pool gaps yet leave $8 \times 10^{-5}$–$5.5 \times 10^{-3}$ full-network gaps; identical for compressed and uncompressed arms, demonstrating that the pool, not the representation, binds accuracy.]

## 6.3 Scenario-reuse protocol with work accounting (P6)

[Frozen compressed model across a demand/capacity scenario grid: break-even at scenario 4 including setup; per-scenario speedups 2.0–3.8× growing with perturbation; measurable staleness

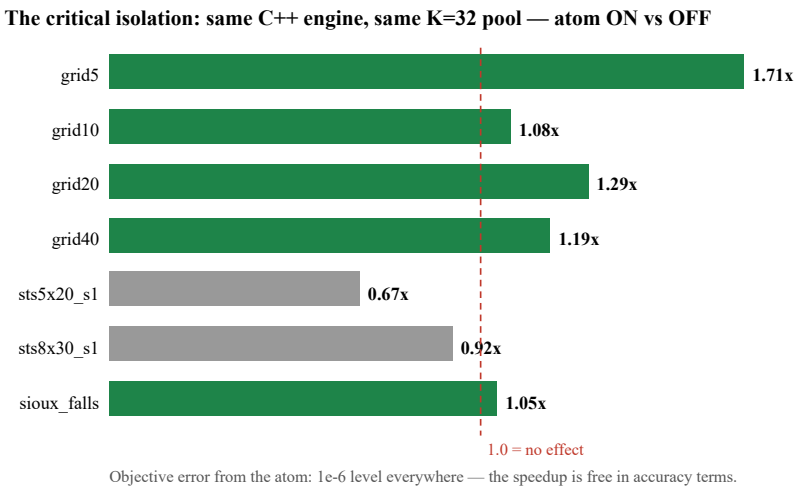


**Figure 2:** Same-kernel isolation: one engine, one pool, compression on/off (P4). The speedup is attributable to representation alone.

threshold near ±20–30% demand drift. Controlled origin-template experiment (B0/L1/L2): the adaptive variant reproduces the reference solution with two templates but cannot beat a 7–9-sweep bush worker on single-origin DAGs — the laboratory reports boundaries, not only successes (P5).]

# 7 Practical Applications

[Agencies: verifying vendor claims with taplab verify; researchers: attribution of performance to algorithm vs. implementation; developers: regression laboratory; educators: analytical instances with exact answers (Braess reproduces 82.25 vs. 66.00 minutes at equilibrium).]

# 8 Conclusions and Ongoing Refinements

[Summary; scheduled refinements: unified C++ kernel with pybind11 bindings, cyclic-network origin workers, Dolan–Moré performance profiles, schedule-based transit track.]

# Acknowledgments

[To be added.]

# Author Contributions

[Per TRB requirements.]

# References


Bar-Gera, H. 2002. Origin-based algorithm for the traffic assignment problem. *Transportation Science* 36 (4): 398–417.

Beiranvand, V., W. Hare, and Y. Lucet. 2017. Best practices for comparing optimization algorithms. *Optimization and Engineering* 18: 815–848.

Dial, R. B. 2006. A path-based user-equilibrium traffic assignment algorithm that obviates path storage and enumeration. *Transportation Research Part B* 40 (10): 917–936.

Dolan, E. D., and J. J. Moré. 2002. Benchmarking optimization software with performance profiles. *Mathematical Programming* 91: 201–213.

Gentile, G. 2014. Local user cost equilibrium: a bush-based algorithm for traffic assignment. *Transportmetrica A* 10 (1): 15–54.

Halko, N., P.-G. Martinsson, and J. A. Tropp. 2011. Finding structure with randomness. *SIAM Review* 53 (2): 217–288.

Jayakrishnan, R., W. K. Tsai, J. N. Prashker, and S. Rajadhyaksha. 1994. A faster path-based algorithm for traffic assignment. *Transportation Research Record* 1443: 75–83.

Perederieieva, O., M. Ehrgott, A. Raith, and J. Y. T. Wang. 2015. A framework for and empirical study of algorithms for traffic assignment. *Computers & Operations Research* 54: 90–107.

Xie, J., and C. Xie. 2015. Origin-based algorithms for traffic assignment: algorithmic structure, complexity analysis, and convergence performance. *Transportation Research Record* 2498: 46–55.

Zhou, X., P. Li, Y. Liu, Y. Li, and D. Bertsekas. 2026. Compressed traffic assignment with the augmented Lagrangian method. Preprint, arXiv:2604.23101.